# Chapitre 5 :
# Classes de complexité

Sources :
- Carton : Langages formels: Calculabilité et complexité
- Wolper : Introduction à la calculabilité (/!\ Pas très bien traduit)
- Garey-Johnson : Computers and intractability : a guide to the theory of NP-completness

## Table des matières


**I Modèles de calcul** ..... **1**
- I.1 Taille d'une entrée ..... 1
- I.2 Temps d'exécution ..... 2

**II Classes de complexité en temps** ..... **2**
- II.1 Classes de complexité en temps déterminisite ..... 2
- II.2 Certificats et vérificateurs ..... 4

**III Complétude** ..... **7**
- III.1 Réduction polynomiale ..... 7
- III.2 Problèmes NP-complets ..... 8


# I Modèles de calcul

## I.1 Taille d'une entrée

**Définition**

La **taille** d'une instance d'un problème (de décision) est l'espace mémoire nécessaire à sa représentation, c'est à dire le nombre de caractères nécessaire pour encoder cette instance.

***Remarque***

On se satisfait d'ordres de grandeur de la taille dans les estimations de complexité.

**Avertissement**

Il y a deux cas à distinguer pour caractériser la taille d'un entier :

- Si les entiers sont bornés, on peut considérer que leur taille est en $O(1)$.
- Si les entiers ne sont pas bornés, on peut considérer que la taille de leur représentation en base $b$ est en $\log_b(n)$, donc que leur taille est en $O(\log(n))$.

Dans le second cas, un algorithme en $O(n)$ si $n$ est un entier non borné est ***exponentiel en la taille de son entrée***.

**Exemple 1**:
Pour le problème PREMIER, la taille de l'entrée est $\log n$
Pour le problème PGCD la taille de l'entrée est $\log a + \log b$, donc $\log ab$

## I.2 Temps d'exécution

**Définition**

Une **valeur atomique** est une valeur alphanumérique représentable dans notre modèle de calcul sur un nombre fixe de bits.

**Exemple 2**:
Un entier sur 32 bits est une valeur atomique.
Un entier sans plus de précisions n'est pas une valeur atomique

**Définition**

Une **opération élémentaire** est une opération s'exécutant en temps constant.

Dans notre modèle de calcul, les opérations élémentaires sont les lecture, écritures, comparaisons et opérations arithmétiques sur des valeurs atomiques

Si A est un algorithme, le **temps d'exécution** de A sur une entrée $x$ est le nombre d'opérations élémentaires faites par A lors de son exéution sur $x$.

# II Classes de complexité en temps

## II.1 Classes de complexité en temps déterminisite

**Définition**

Une **classe de complexité** est un ensemble de problèmes de décision partageant un point commun vis à vis de la complexité des algorithmes qui les résolvent.

**Définition**

Si $t : \mathbb{N} \to \mathbb{N}$, on note $\mathrm{DTIME}(t(n))$ l'ensemble des problèmes de décision $\mathscr{P}$ tel qu'il existe un algorithme déterministe qui résout $\mathscr{P}$ et ayant une complexité en $\mathrm{O}(t(n))$.

Si T est un ensembles de fonctions de $\mathbb{N} \to \mathbb{N}$, on note
$\mathrm{DTIME}(\mathrm{T}) = \bigcup_{t \in \mathrm{T}} \mathrm{DTIME}(t(n))$

**Définition**

$\mathrm{P} = \bigcup_{k \in \mathbb{N}} \mathrm{DTIME}(n^k) = \mathrm{DTIME}(n^{\mathrm{O}(1)})$ est donc l'ensemble des problèmes de décision résolubles en temps polynomial

**Avertissement**

- Si $\mathscr{P}$ est un problème de décision et qu'on trouve un algorithme exponentiel pour résoudre $\mathscr{P}$, on ne peut pas conclure $\mathscr{P} \notin P$
- Le fait que $\mathscr{P}$ soit dans P ne veut pas dire qu'*en pratique* on saura résoudre $\mathscr{P}$ de manière efficace
- Ce sont les problèmes de décision qui sont dans P, pas les fonctions ni les algorithmes.

***Remarque***

On notre EXP la classe des problèmes résolubles en temps exponentiel.

**Exemple 3**:

$$\mathrm{PGCD} : \begin{cases} \underline{\text{Entrée :}}\ (a,b) \in N \\ \underline{\text{Solution :}}\ d \in \mathbb{N} \text{ tel que } d|a \text{ et } \ d|b \\ \underline{\text{Optimisation :}} \text{ maximiser } d \end{cases}$$

Ce problème est résolu par l'algorithme d'euclide d'Euclide en temps $\mathrm{O}(\log(a+b))$ ce qui est bien polynomial en la taille $\log a + \log b$ de l'entrée

$$\text{PREMIER} : \begin{cases} \underline{\text{Entrée :}}\ n \in \mathbb{N} \\ \underline{\text{Question :}}\ n \in \mathbb{P}\ ? \end{cases}$$

Ce problème est dans EXP via l'algorithme naïf en $\mathrm{O}(\sqrt{n})$.
En fait il est dans P via l'algorithme AKS.

$$\text{PCC} : \begin{cases} \underline{\text{Entrée :}}\ G = (S, A) \text{ un graphe non pondéré ; } (s, t) \in S \\ \underline{\text{Sortie :}}\ \text{un chemin } (s, s_0, ..., s_k, t) \\ \underline{\text{Optimisation :}}\ \text{minimiser k} \end{cases}$$

Est résoluble via un parcours en largeur en O(|S| + |A|) donc est dans P

$$\text{ARRET_BORNE} : \begin{cases} \underline{\text{Entrée :}}\ \text{Une fonction } f\text{, un argument } x \text{ et un entier } k \\ \underline{\text{Question :}}\ \text{L'exécution de } f \text{ sur } x \text{ termine-t-elle en moins de } k \text{ étapes ?} \end{cases}$$

L'algorithme consistant à simuler l'exécution de $f$ sur $x$ pendant $k$ étapes dans une machine universelle a une complexité en $\mathrm{O}(k) = \mathrm{O}(e^{\log(k)})$. Donc ce problème est dans EXP.
Est-il dans P ?

## II.2 Certificats et vérificateurs

**Idée**

P = {Problèmes résolubles en temps polynomial}
NP = {Problèmes vérifiables en temps polynomial}

**Rappel**

Si A est un problème de décision et $I_A$ est son ensemble d'instances, on note

$$P_A : \begin{cases} I_A \to \mathbb{B} \\ x \mapsto \begin{cases} \text{vrai si } x \text{ est positive pour A} \\ \text{faux sinon} \end{cases} \end{cases}$$

**Définition**

La classse NP est l'ensemble des problèmes de décision A tel qu'il existe :

- Un problème $B \in P$ tel que $I_B = I_A \times \{0,1\}^*$
- Une fonction polynomiale $f$ telle que pour tout $x$ instance de A,
  $P_A(x) \iff \exists c \in \{0,1\}^*$ tel que $|c| \leq f(|x|)$ et $P_B((x,c))$

**Définition**

- Une algorithme polynomial qui résout B s'appelle un **vérificateur**
- Un indice permettant de confirmer la positivité d'une instance s'appelle un **certificat**

**Exemple 4**: SAT est dans NP

On note :

$$\text{VAL} : \begin{cases} \underline{\text{Entrée :}} \text{ Une formule du calcul propositionnel } \varphi \text{ et une valuation } v \\ \underline{\text{Question :}} \text{ Est-ce que } v \text{ satisfait } \varphi \end{cases}$$

$f : n \mapsto n$

$f$ est polynomial et
VAL $\in P$ car on peut le résoudre avec un algorihtme linéaire en $|\varphi|$.

$\varphi$ est satisfiable si et seulement s'il existe une valuation $v$ telle que $v(\varphi) = 1$
$\varphi$ est une instance positive de SAT si et seulement s'il existe un indice $v$ de taille $f(|\varphi|)$ tel que $(\varphi, v)$ est positive pour VAL.

**Méthode**

En pratique, on n'utilise pas ce formalisme mais, quand on veut faire une preuve que $\mathscr{P}$ est fans NP, on doit mentionner 3 points :

1. Indiquer si une instance $i$ est positive, quel objet $o$ permet de le vérifier
2. Vérifier que la taille de l'objet $o$ est polynomial en $|i|$
3. Expliquer comment la connaissance de $o$ permet de savoir si $i$ est bien positive et justifier que la méthode utilisée est polynomiale en $|(i,o)|$

**Exemple 5**:

$$\text{CH} : \begin{cases} \underline{\text{Entrée :}} \text{ Un graphe non orienté G = (S,A)} \\ \underline{\text{Question :}} \text{ Est-ce qu'il y a un cycle dans G qui passe une et une seule fois par chaque sommet ?} \end{cases}$$

est dans NP

Pour toute instance positive de CH, il existe une suite C de sommets de taille inférieure à |S| sans doublons, qui contient tous les sommets s et qui représente un cycle dans G.

Or, étant donné un graphe G et une suite C de moins de |S| sommets, on peut vérifier en temps polynomial en |G| si G vérifie les propriétés voulues :

- On vérifie que C est un chemin en vérifiant que deux sommets consécutifs de C sont liés par une arête (O(|S|) ou O($|S|^2$) selon la représentation du graphe)
- On vérifie que le premier sommet est connecté au dernier
- On vérifie que C est dans doublons en O($|S|^2$)
- On vérifie que |C| = |S| pour s'assurer que tous les sommets de G sont dans C

**Exemple 6**:

$$\text{CLIQUE} : \begin{cases} \underline{\text{Entrée :}} \text{ Un graphe G = (S,A) non orienté, } k \in \mathbb{N} \\ \underline{\text{Question :}} \text{ Est-ce qu'il existe un sous-graphe de G complet à } k \text{ sommets} \end{cases}$$

est dans NP.

Pour toute instance de CLIQUE il existe un ensemble X de sommets tels que pour tout $(x, y) \in X^2$, $(x, y) \in A$ et tel que |X| = k.
On remarque que |X| ≤ |S|.

De plus, étant donné un grpahe G = (S,A) et une semble X ⊂ S, on peut vérifier que X est une clique de taille $k$ dans G en temps polynomial :

- On vérifie qu'il y a $k$ éléments dans X
- On vérifique que pour pour tout $(x, y) \in X^2, (x, y) \in A$, ce qui peut se faire en O($|X|^2$) = O($|S|^2$) si on représente le graphe par liste d'adjacence.

**Proposition 1**: P ⊂ NP

**Démonstation**: En exercice

🔥 ***Remarque***

On pense que cette inclusion est stricte mais c'est une quesiton ouverte

**Avertissement**

Si dans une copie vous écrivez quelque chose que P = NP c'est qu'il y a sûrement une merde quelque part.

NP veut dire « non-deterministic polynomial » et PAS « non-polynomial »

# III Complétude

## III.1 Réduction polynomiale

**Définition**

Si A et B sont des problèmes de décision, on dit que **A se réduit polynomialement à B** (noté A $\leq$ B) si et seulement s'il existe une fonction $f : I_A \to I_B$ calculable en temps polynomial et telle que pour tout $x \in I_A, P_A(x) = P_B(f(x))$

**Exemple 7**: STABLE $\leq$ CLIQUE avec :

$$\text{STABLE} : \begin{cases} \underline{\text{Entrée :}} \text{ Une graphe non orienté G} = \text{(S,A)}, k \in \mathbb{N} \\ \underline{\text{Question :}} \text{ Est-ce qu'il existe } X \subset S \text{ de taille } k \text{ tq } \forall (x,y) \in X^2, (x,y) \notin A \end{cases}$$

Soit (G = (S, A), $k$) une instance de STABLE.
On peut la transofrmer en ($\overline{G}$ = (S, $\overline{A}$), $k$) qui est une instance de CLIQUE, avec $\overline{A}$ le complémentaire de $A$ dans $\delta^2$.

Cette transformation peut s'effectuer en temps O(|S|$^2$) si G est représenté par matrice d'adjacence, ce qui est polynomial en |G|.

De plus, (G = (S, A), $k$) est une instance positive pour STABLE si et seulement si $\overline{G}$ = (S, $\overline{A}$), $k$) est une instance positive pour CLIQUE.

**Proposition 2**: Si $A \leq B$ :
- Si $B \in P$ alors $A \in P$
- Si $B \in$ NP alors $A \in$ NP

**Démonstation**: Comme $B \in P$, il existe une fonction calculable en temps polynomial $f_B : I_B \to \mathbb{B}$ qui résout B.
Comme $A \leq B$, il existe une fonction $f : I_A \to I_B$ calculable en temps polynomial et telle que pour toute instance $x \in I_A$, $P_A(x) \iff P_B(f(x))$ $(\star)$

Pour résoudre A on peut donc calculer $f(x)^{\mathbf{1}\cdot}$ et résoudre le problème B avec $f_B$ sur l'instance $f(x)^{\mathbf{2}\cdot}$. Ceci donne une résolution correcte de A grâce à $(\star)$

Cet algorithme s'exécute en temps polynomial en $|x|$ car :

1. s'exécute en temps polynomial en $|x|$ par définition d'une réduction.
2. s'exécute en temps polynomial car :
   - $|f(x)|$ est polynomial en $|x|$ car la complexité spatiale d'une fonction est plus petite que sa complexité en temps et $f$ a une complexité polynomiale en $|x|$
   - $f_B$ s'exécue en temps polynomial en $|f(x)|$ mais comme cette taille est polynomiale en $|x|$, $f_B(f(x))$ s'exécute bien en temps polynomial en $|x|$

**Proposition 3**: Cette relation de réduction est transitive et refléxive

**Démonstation**: Laissée en exercice au lecteur

## III.2 Problèmes NP-complets

**Définition**

Si A est un problème de décision :
- A est **NP-difficile** si pour tout problème $B \in$ NP, $B \leq A$
- A est **NP-complet** si'il est NP-difficile et dans NP.

**Proposition 4**: Si A est NP-dfficile et $A \in P$ alors P = NP

**Démonstation**: On a déjà $P \subset$ NP, il suffit donc de montrer l'inclusion réciproque.

Soit $B \in$ NP, comme A est NP-difficile, alors $B \leq A$.
Or, par la proposition 2, B est alors dans P.

**Avertissement**

Si on montre qu'un alogrithme qui résout un problème NP-difficile est polynomial, on a montré que P = NP, un peu sus.

**Théorème de Cook-Levi (1)**: SAT est NP-complet

**Démonstation**: On a déjà montré que SAT est dans NP. De plus SAT est NP-difficile (hors-programme)

🏁 **Méthode**

Pour montrer qu'un problème $\mathscr{P}$ est NP-complet :
1. On montre qu'il est dans NP
2. On trouve un problème NP-complet $C$ tel que $C \leq \mathscr{P}$

**Proposition 5**: Si $A \leq B$, $A$ est NP-difficile et $B \in$ NP
Alors $B$ est NP-complet

**Démonstation**:
1. $B \in$ NP par hypothèse
2. Soit $C \in$ NP. Comme A est NP-difficile, $C \leq A$
   Comme $A \leq B$ et par transitiviété de $\leq$, $C \leq B$
   Pourt tout $C \in$ NP, $C \leq B$ : c'est la définition de $B$ est NP-difficile.

**Théorème (2)**: **3SAT est NP-complet** avec

$$\text{3SAT} : \begin{cases} \underline{\text{Entrée}} : \text{Une formule } \varphi \text{ du calcul propositionnel, sous FNC} \\ \qquad\qquad \text{avec au plus trois littéraux par clause} \\ \underline{\text{Question}} : \text{Est-ce que } \varphi \text{ est satisfiable ?} \end{cases}$$

**Démonstation**:
- 3SAT $\in$ NP, même preuve que pour SAT
- Pour montrer que 3SAT est NP-difficile, on montre que SAT se réduit polynomialement à 3-SAT. (Comme SAT est NP-complet, cela montrera ce qu'on veut.)

Soit $\varphi$ une instance de SAT. On peut supposer que les seuls connecteurs dans $\varphi$ sont $\vee$ et $\neg$ quitte à remplacer les autres en faisant augmenter la taille de $\varphi$ de manière linéaire.

On note $\mathrm{SF}(\varphi)$ l'ensemble des sous-formules de $\varphi$
et $\mathrm{V}(\varphi)$ l'ensemble des variables dans $\varphi$.

$\forall \psi \in \mathrm{SF}(\varphi)$, on introduit une nouvelle variable $x_\psi$.
Enfin, on note $\mathrm{T}(\varphi) = x_\psi \wedge \bigwedge_{\psi \in \mathrm{SF}(\varphi) \setminus V(\varphi)} t(\psi)$ avec :

- $t(\neg \ \psi) = \left(x_{\neg\psi} \vee x_\psi\right) \wedge \left(\neg x_{\neg\psi} \vee \neg x_\psi\right)$
- $t(\psi_1 \vee \psi_2) = \left(\neg x_{\psi_1 \vee \psi_2} \vee x_{\psi_1} \vee x_{\psi_2}\right) \wedge \left(\neg x_{\psi_1} \vee x_{\psi_1 \vee \psi_2}\right) \wedge \left(\neg x_{\psi_2} \vee x_{\psi_1 \vee \psi_2}\right)$

1. T transforme une instance de SAT en une instance de 3SAT.
2. $\mathrm{T}(\varphi)$ est calculable en temps polynomial en $|\varphi|$.
   Calculer $\mathrm{SF}(\varphi)$ et $\mathrm{V}(\varphi)$ se fait en $\mathrm{O}(|\varphi|)$ en parcourant l'arbre syntaxique de la formule.

3. $\varphi$ est satisfiable si et seulement si $\mathrm{T}(\psi)$ est satisfiable.

($\Longrightarrow$) On remarque que :

- $t(\neg\psi) \equiv \left(\neg x_\psi \Leftrightarrow x_{\neg\psi}\right)$

En effet,

$$\begin{aligned}\left(\neg x_\psi \Leftrightarrow x_{\neg\psi}\right) &\equiv \left(\neg x_\psi \Rightarrow x_{\neg\psi}\right) \wedge \left(x_{\neg\psi} \Rightarrow \neg x_\psi\right)\\ &\equiv \left(\neg x_{\neg\psi} \vee \neg x_\psi\right) \wedge \left(x_\psi \vee x_{\neg\psi}\right)\\ &\equiv t(\neg\psi)\end{aligned}$$

- $t(\psi_1 \vee \psi_2) \equiv \left(x_{\psi_1} \vee x_{\psi_2} \Leftrightarrow x_{\psi_1 \vee \psi_2}\right)$

Ainsi, si $\varphi$ est satisfiable par $v$, on construit $\widetilde{v}$ qui satisfait $T(\varphi)$ comme suit : $\forall \psi \in \mathrm{SF}(\psi), \widetilde{v}\left(x_\psi\right) = v(\psi)$.

$\widetilde{v}$ satisfait bien $T(\varphi)$
- $\widetilde{v}\left(x_\varphi\right) = v(\varphi) = 1$ car $v$ satisfait $\varphi$
- $\widetilde{v}(t(\neg\psi)) = \widetilde{v}\left(\neg x_\psi \Leftrightarrow x_{\neg\psi}\right) = 1$
  car $\widetilde{v}\left(x_{\neg\psi}\right) = v(\neg\psi) = \overline{v(\psi)} = \widetilde{v}\left(\neg x_\psi\right)$
- $\widetilde{v}(t(\psi_1 \vee \psi_2)) = \ldots$

($\Longleftarrow$) Réciproquement, si $T(\varphi)$ est satisfiable par $\widetilde{v}$, la valuation $v$ suivante satisfait $\varphi$ : $v(p) = \widetilde{v}\left(x_p\right)$

Comme $\widetilde{v}$ satisfait $T(\varphi)$, $\widetilde{v}\left(x_\varphi\right) = 1$ en particulier.
Ainsi, $v(\varphi) = 1$ aussi car pour toute formule $\gamma, \widetilde{v}\left(x_\gamma\right) = v_\gamma$.

On a montré que SAT $\leq$ 3SAT. Comme SAT est NP-difficile, 3SAT aussi.
Comme 3SAT $\in$ NP, 3SAT est NP-complet.

🔥 ***Remarque***

CNF-SAT et $k$SAT pour $k \geq 3$ sont NP-complet

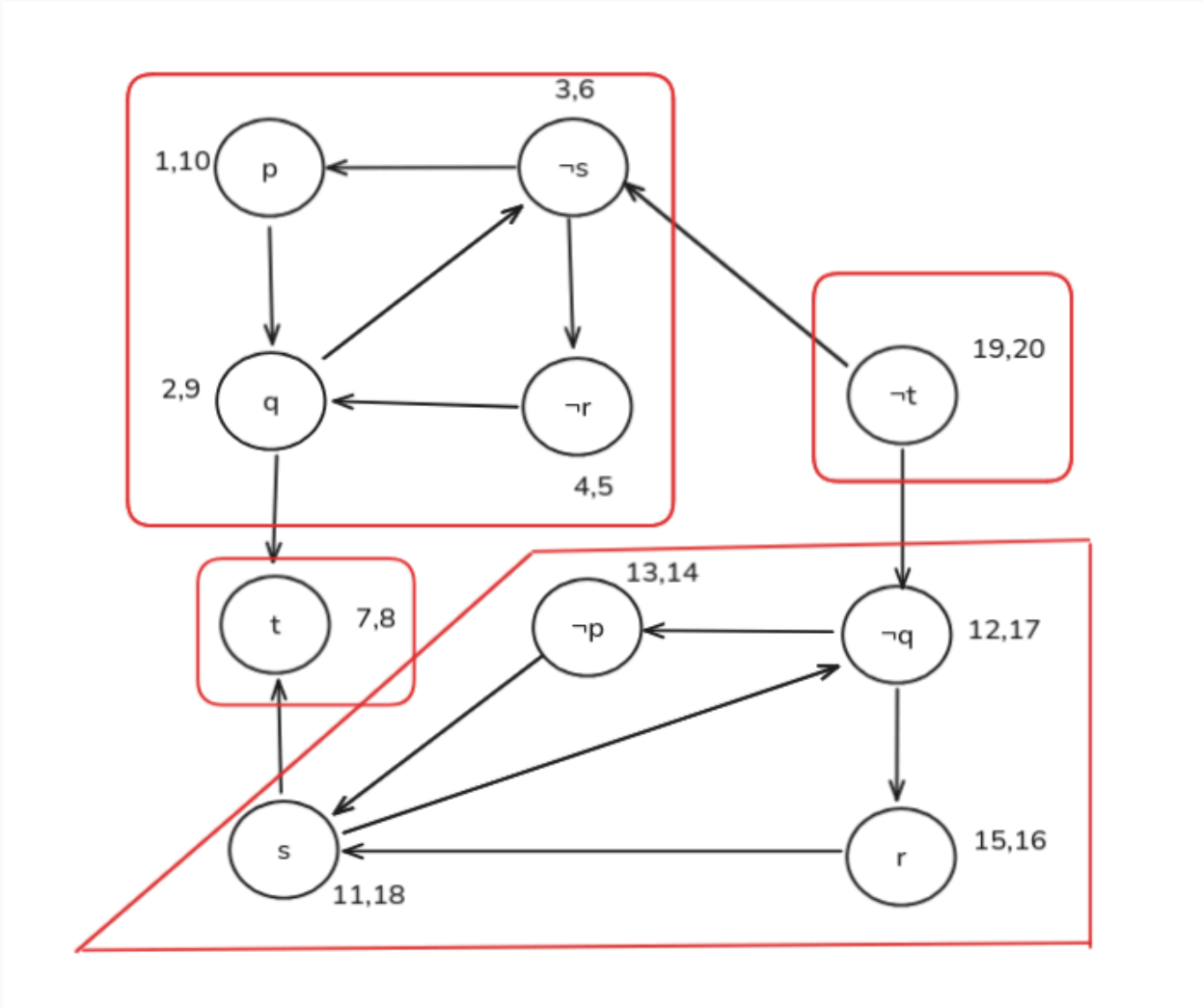
**Théorème (3)**: **2SAT $\in$ P** avec :

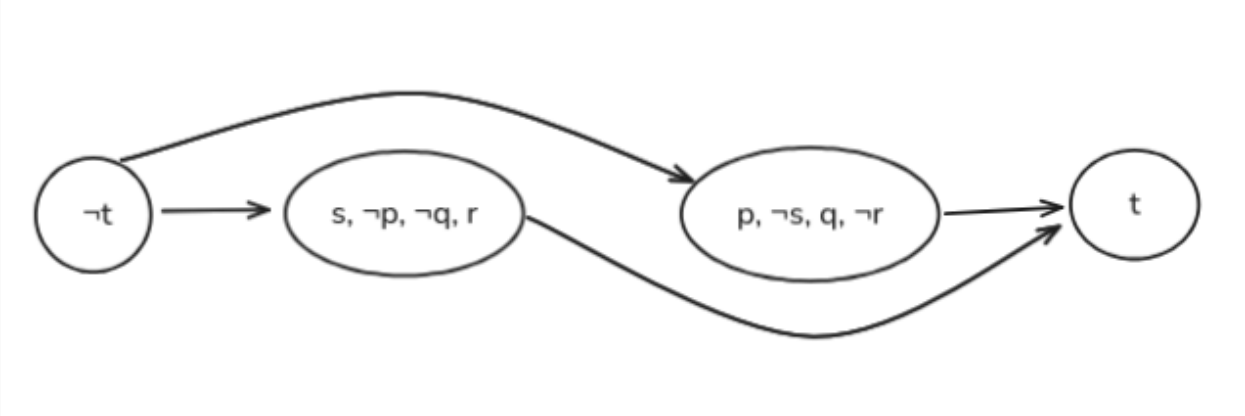
$$\text{2SAT} : \begin{cases} \underline{\text{Entrée}} : \text{Une formule } \varphi \text{ du calcul propositionnel, sous FNC} \\ \qquad\qquad \text{avec exactement deux littéraux par clause} \\ \underline{\text{Question}} : \text{Est-ce que } \varphi \text{ est satisfiable ?} \end{cases}$$

**Démonstation**: Voici un algorithme qui résout 2SAT en temps polynomial.

1. Calculer $\mathrm{Var}(\varphi)$ l'ensemble des variables de $\varphi$
2. Construire un graphe $G_\varphi = (\mathrm{V}, \mathrm{E})$ avec
   $\mathrm{V} = \mathrm{Var}(\varphi) \cup \{\neg x \mid x \in \mathrm{Var}(\varphi)\}$
   $\mathrm{E} = \{(\neg e_1, e_2) \mid e_1 \vee e_2 \text{ une clause de } \varphi\} \cup \{(\neg e_2, e_1) \mid e_1 \vee e_2 \text{ une clause de } \varphi\}$
3. Calculer les CFC de $G_\varphi$
4. Pour chaque CFC C trouvée, on regarde s'il existe $x \in \mathrm{Var}(\varphi)$ tel que $x$ et $\neg x$ sont sommets de C.
   Si oui : $\varphi$ n'est pas satisfiable
   Si non : $\varphi$ est satisfiable et on peut construire une valuation $v$ satisfiant $\varphi$ en assignant à vrai tous les littéraux dont la valeur de vérité est encore inconnu CFC par CFC en remontant l'ordre topologique.

**Exemple 8**: Voici $G_\varphi$ pour $\varphi = (\neg p \vee q) \wedge (\neg r \vee s) \wedge (p \vee s) \wedge (\neg q \vee t) \wedge (r \vee q) \wedge (t \vee \neg s) \wedge (\neg s \vee \neg q)$






Ainsi, $\varphi$ est satisfait pour $v(t) = 1, v(p) = 1, v(q) = 1, v(s) = 0, v(r) = 0$

L'algorithme est polynomial en la taille de $\varphi$ :
1. est en $\mathrm{O}(|\varphi|)$ en parcourant $\varphi$
2. est en $\mathrm{O}(|\mathrm{Var}(\varphi)| + \text{nombre de clauses}) = \mathrm{O}(|\varphi|)$ et produit un arc qui a $\mathrm{O}(|\varphi|)$ sommets et $\mathrm{O}(|\varphi|)$ arcs.
3. est en $\mathrm{O}(|\varphi| + |\varphi|)$ par Kosaraju
4. esg en $\mathrm{O}(|\mathrm{Var}(\varphi)|) = \mathrm{O}(|\varphi|)$

**Démonstation**: ***Suite de la démonstration***

Cet algorihtme résout 2SAT, pour montrer ceci on montre que $\varphi$ est satisfiable si et seulement si pour toute variable $x$ de $\varphi$, on n'as jamais $x$ et $\neg x$ dans la même CFC de $G_\varphi$

($\Longrightarrow$) Comme $\varphi$ est satisifable, il existe une valuation $v$ telle que $v(\varphi) = 1$.
Montrons par récurrence sur $n \in \mathbb{N}, H(n)$ : s'il existe un chemin de longueur $n$ de $l$ vers $l'$ dans $G_\varphi$ et que $v(l) = 1$ alors $v(l') = 1$

Si $n = 0$ c'est direct.

Soit $n \in \mathbb{N}$ tel que $H(n)$ soit vrai.
Soit $l$ et $l'$ deux littéraux liés par un chemin de longueur $n + 1$ avec $v(l) = 1$.

$l \to l'' \rightsquigarrow l$.
Comme $(l, l'')$ est une arête, la construction de $G_\varphi$ dit que $\varphi$ contient la clause $\neg l \vee l''$. Cette clause est satisfaite par $v$ mais ne satisfait pas $\neg l$, donc $v(l'') = 1$.
On n'as plus qu'à appliquer l'hypothèse de récurrence sur $l'' \rightsquigarrow l$.

Si on avait $x$ et $\neg x$ dans la même CFC, alors
- Si $v(x) = 0, v(\neg x) = 1$ et comme $\neg x \rightsquigarrow x$ on aurait $v(x) = 1$.
- Si $v(x) = 1$, comme $x \rightsquigarrow \neg x$, on aurait $v(\neg x) = 1$ donc $v(x) = 0$.

($\Longleftarrow$) On note $C_1, ..., C_k$ les CFC de $G_\varphi$ dans un ordre topologique. On construit une valuation $v$ en fixant à vrai les littéraux non encore assignés CFC par CFC en remonant l'ordre topologique.

Montrons que $v$ satisfait $\varphi$.

$\varphi$ est bien défini car assigner vrai à $x$ et $\neg x$ indiquerait que ces deux littéraux sont dans la même CFC, ce qui est exclu.

Montrons que $v$ satisfait $l \vee l'$ une clause de $\varphi$.
$l \vee l' \equiv (\neg l \Rightarrow l') \wedge (\neg l' \Rightarrow l)$

Notons $x$ l'un des premiers littéraux parmis $\{l, l', \neg l, \neg l'\}$ à apparaître dans une CFC en remontant l'ordre topologique.

- Si $x = l$, alors par construction de $v$, $v(l) = 1$ et donc $v(l \vee l') = 1$
- Si $x = \neg l$, alors par construction, $v(l) = 0$ et donc il faut montrer que $v(l') = 1$. Comme $l \vee l'$ est une clause, il y a un arc $\neg l \to l'$, donc $l'$ est dans une CFC plus petite que $\neg l$.
  Ça ne peut pas être une CFC strict plus petite sinon la définition de $x$ est contredite.
  De plus, $\neg l'$ intervient dans une CFC plus grande pour l'ordre topologique (sinon la def de $x$ est contredite).
  Donc, au moment où on consièdre la CFC contenant $l'$, ce littéral n'est pas encore assigné et donc on fixe par construction $v(l') = 1$, donc $v(l \vee l') = 1$.

**Théorème (hors-programme) (4)**:
**Le problème du cycle hamiltonien orienté est NP-complet**

$$\text{CHO} : \begin{cases} \underline{\text{Entrée :}} \text{ Un graphe orienté G} \\ \underline{\text{Question :}} \text{ Existe-t-il un cycle dans G qui passe une et} \\ \qquad\qquad \text{une unique fois par chaque sommet ?} \end{cases}$$

**Démonstation**:

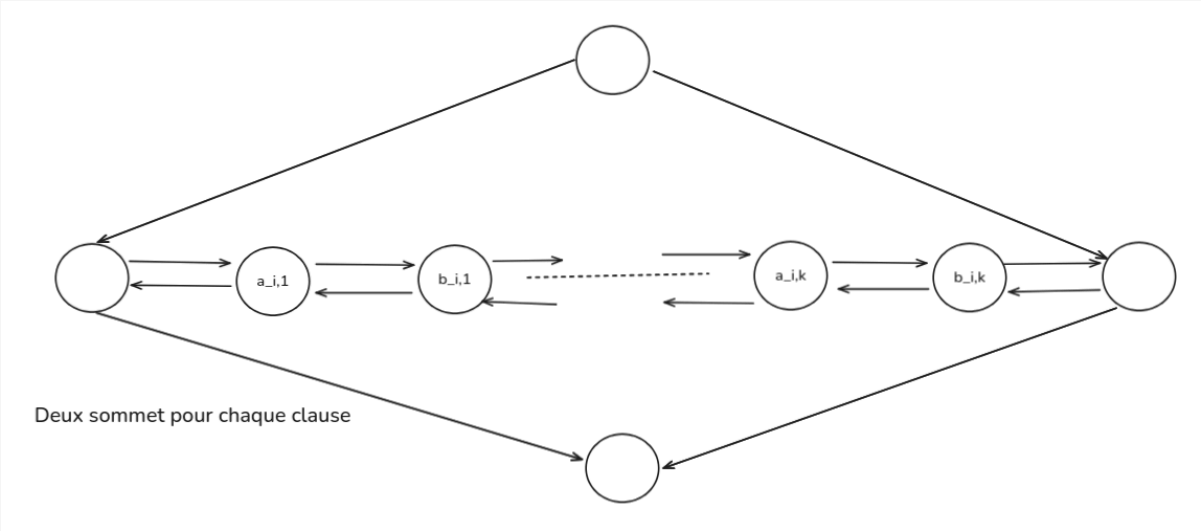
- CHO est dans NP car une instace de CHO est positive si et seulement s'il existe un ensemble $X \subset S$ qui soit un cycle hamiltonien.

  Or, étant donné G = (S,A) et $X \subset S$, vérifier que $X$ est un cycle hamiltonien se fait en temps polynomial en $|G|$ :
  - On vérifie que $X$ encore un chemin (en O($|X|$) si G est encodé par matrice d'adjacence)
  - On vérifie que le premier et derniser sommet de $X$ sont égaux
  - On vérifie que que $X$ contient tous les sommets exactement une fois (en O($|S|$))

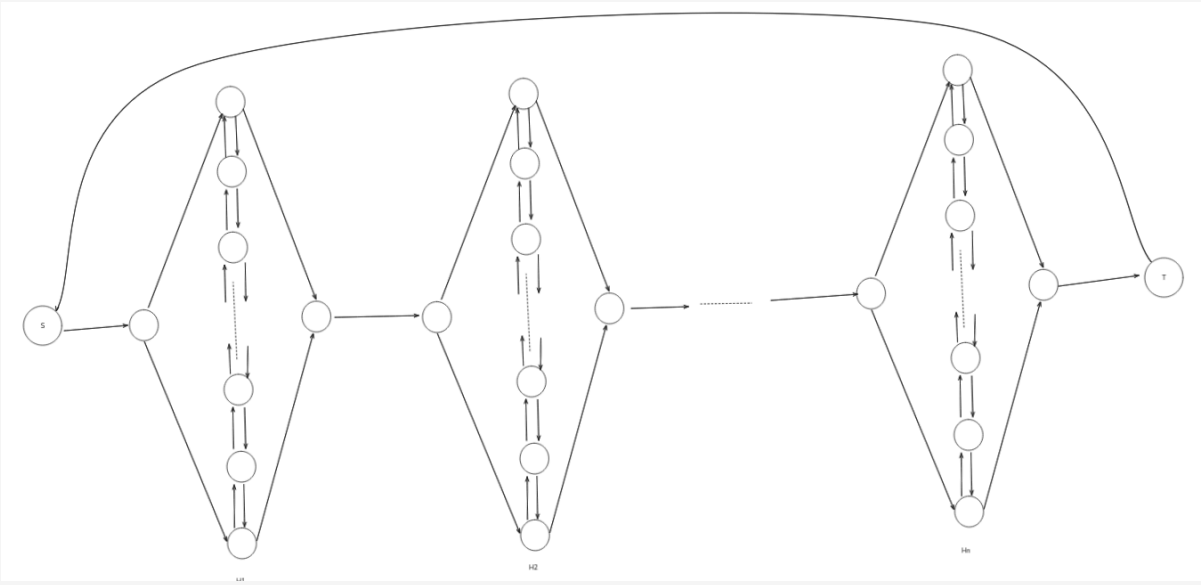
- CHO est NP-difficile par réduction polynomiale de 3SAT en CHO.

  On peut supposer quitte à prétraiter $\varphi$ en temps linéaire qu'aucune close ne fait intervenir un littéral et sa négation.
  Soit $\varphi$ une instance de 3SAT dont on note $C_1, ...C_k$ les clauses et $n$ le nombre de variables qui y interviennent.

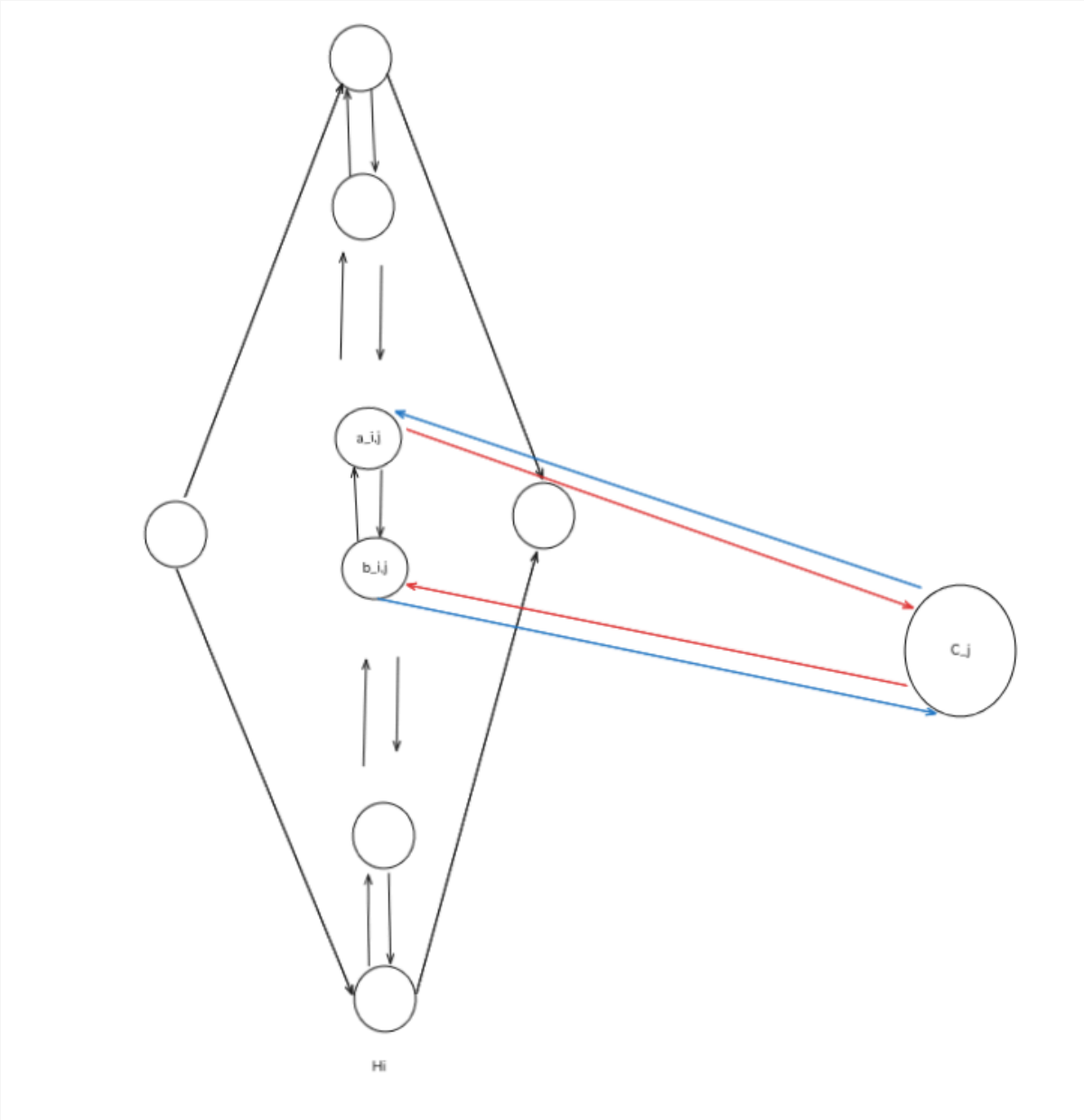
  Pour chaque variable $x_i$ on construit un gadget $H_i$ qui est le graphe suivant :




  On peut alors enchainer les gadgets de la manière suivante :



Enfin, pour chaque clause $C_j$, on ajoute un sommet $c_j$. Pour chaque variable $x_i \in C_j$, on ajoute les arcs rouge si $x_i$ apparaît positivement dans $C_j$ et on ajoute les arcs bleus s'il apparaît négativement.




Calculer $G_\varphi$ se fait en temps polynomial en $|\varphi|$ car il a

- $2 + k + n(4 + 2k)$ sommets avec $k$ clauses, $n$ variables.
  `s et t + clauses + gadgets`
  $= O(|\varphi|^2)$ sommets.
- $n(4 + 2(2k + 1)) + (n + 2) + 6k$
  `arcs dans les gadgets  + connection des gadgets + connection des clauses`
  $= O(|\varphi|^2)$ arcs

De plus, $\varphi$ est satisfiable si et seulement si $G_\varphi$ contient un cycle hamiltonien.

- Si $\varphi$ est satisfiable via une valuation v, alors pour toute clause $C_j$, il existe une variable $x_i$ telle que ($x_i$ intervient positivement dans $C_j$ et $v(x_i) = 0$) ou ($x_i$ intervient négativement dans $C_j$ et $v(x_i) = 0$).

<u>Alors, voici un chemin hamiltonien dans $G_\varphi$ dans entre $s$ et $t$ (qu'on peut donc compléter en cycle hamiltonien via $(t, s)$) :</u>

- Si $v(x_i) = 1$, on parcours $H_i$ de haut en bas en faisant un détour pour chaque $C_j$ non encore visité.
- Si $v(x_i) = 0$, on parcours $H_i$ de bas en haut en faisant un détour à chaque $C_j$ non encore visité.

Réciproquement, s'il existe un cycle hamiltonien dans $G_\varphi$ on peut construire une valuation qui satisfait $\varphi$ en fixant $v(x_i)$ selon le sens du parcours du gadget $H_i$.

**Méthode pour montrer qu'un problème A est NP-complet**

- Montrer que A est dans NP
- Trouver une réduction $B \leq A$ avec B NP-difficile
  - Expliquer comment transformer les instances de B en instance de A.
  - Montrer que cette transformation est polynomiale en la taille des instances
  - Montrer qu'elle préserve la positivité ($\top_B \iff \top_A$)